

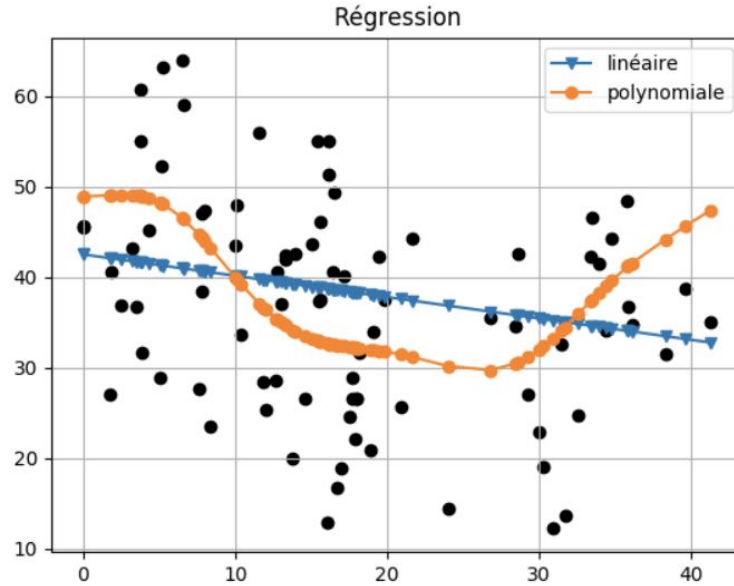
Apprentissage machine

Introduction à l'apprentissage supervisé

Mickaël Bettinelli
(mickael.bettinelli@univ-smb.fr)



Problème des régressions linéaires





Approximer des fonctions

x = données

y = prédictions

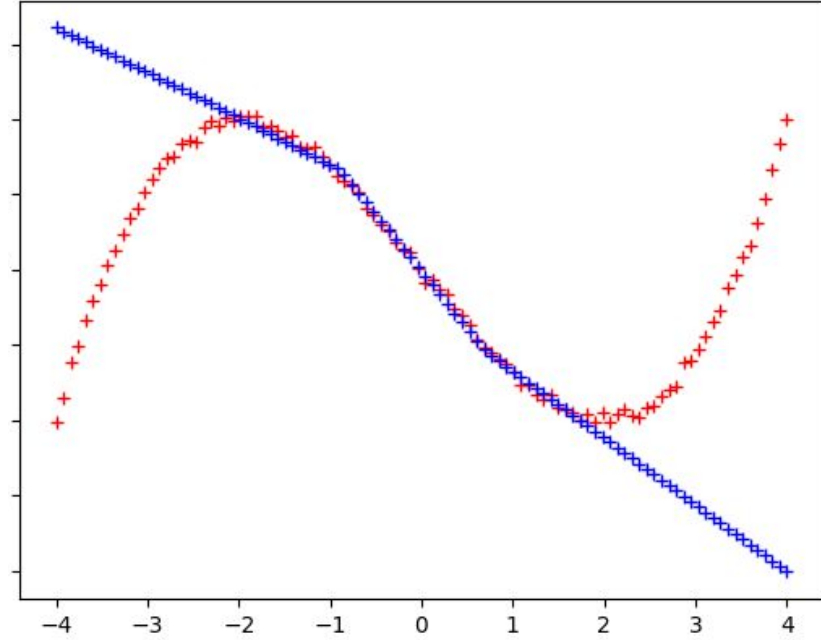
Jeu de données
(entraînement)

Réseau de
neurones

Réseau de
neurones
optimisé

x

y (en bleu)

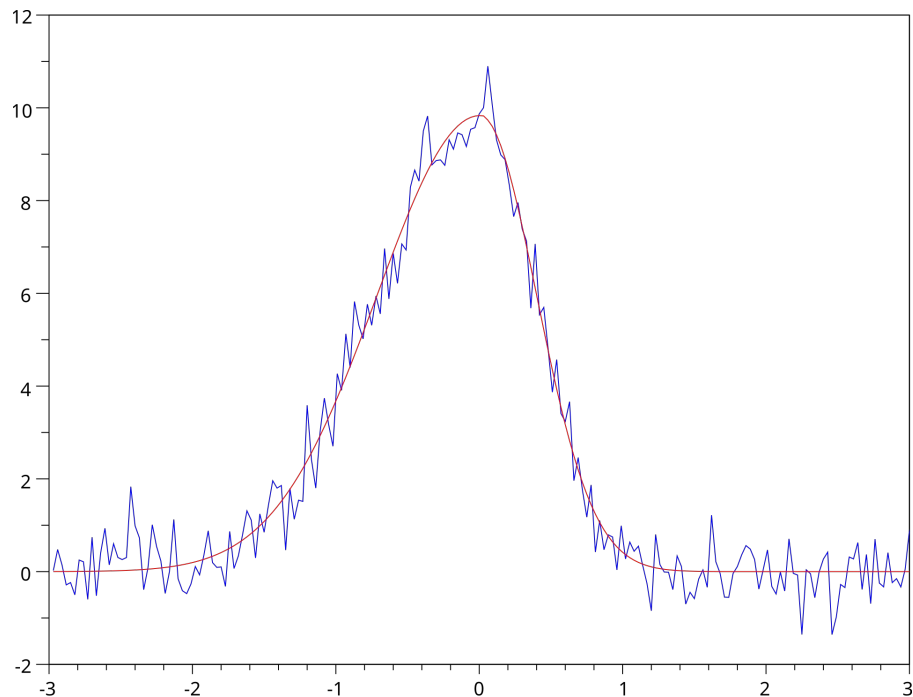




Vue d'ensemble du Deep Learning

Un réseau de neurones est un approximateur de fonctions utilisé de deux manières différentes :

1. Des régressions
2. De la classification

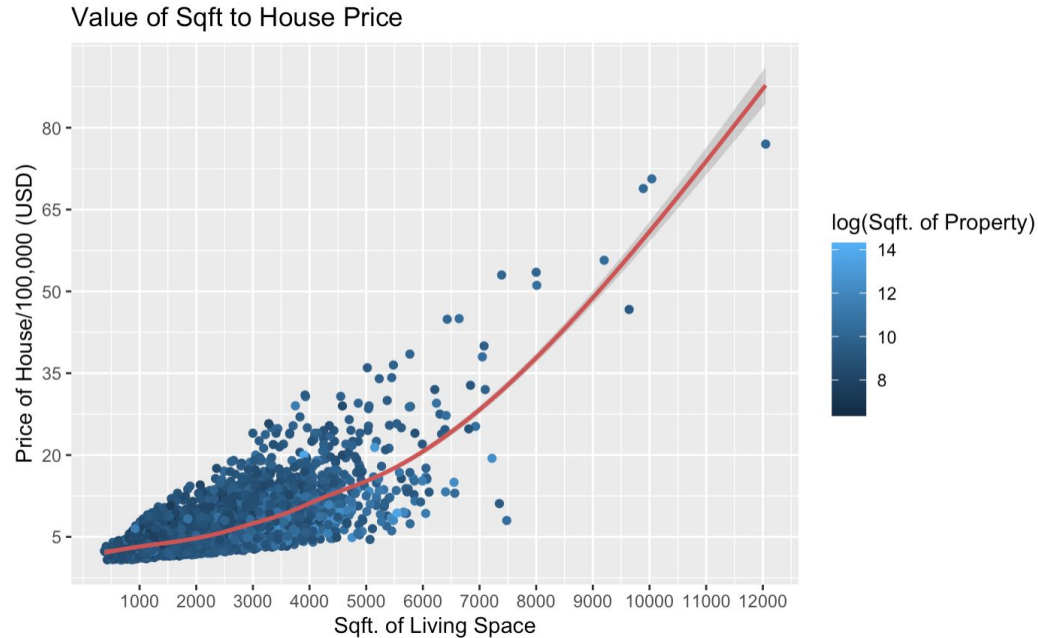




Vue d'ensemble

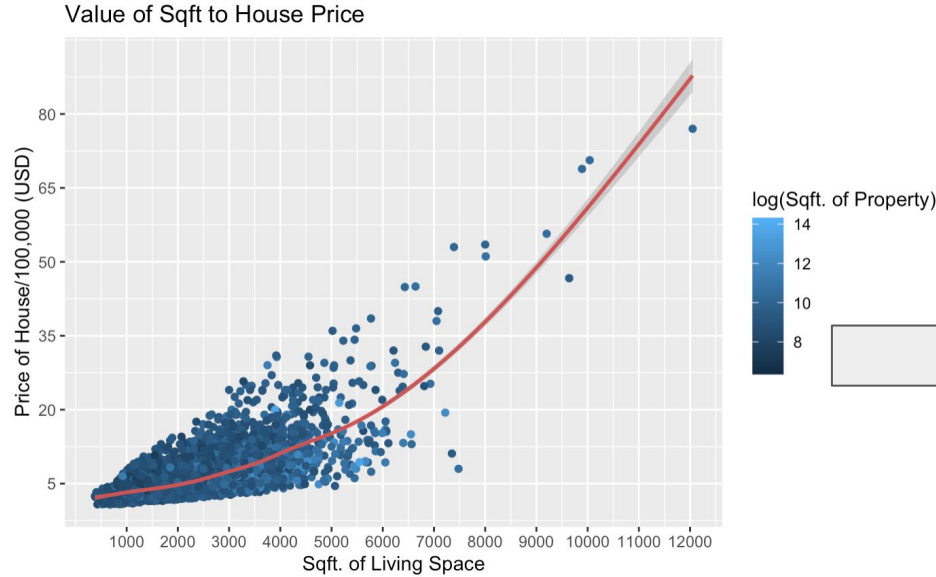
- Régressions :

Exemple. Prix des appartements au m²





Régressions



superficie	orientation	ville	quartier	étage
112	Ouest	A	B	3
...	...	...	...	...

2 dimensions : la superficie et le prix

On peut avoir de nombreuses dimensions !



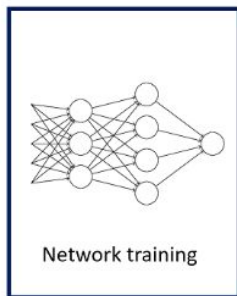
Vue d'ensemble

- Classifications :

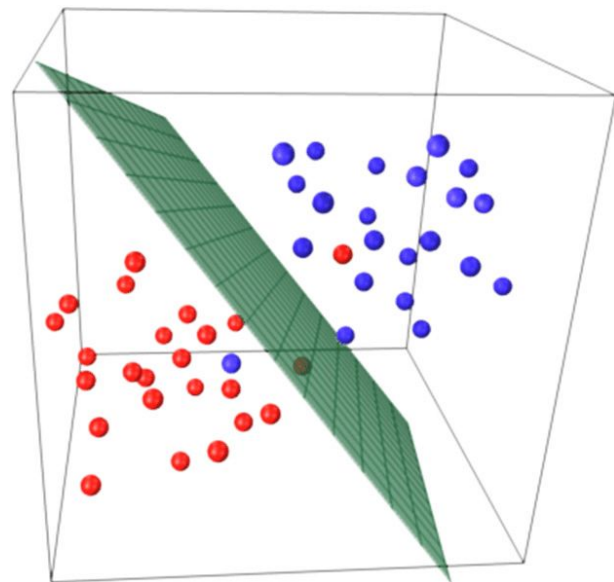
Exemple. Prédiction des chiffres sur des images

0 0 0 0 0 0 0 0 0 0 0 0 0
1 1 1 1 1 1 1 1 1 1 1 1 1
2 2 2 2 2 2 2 2 2 2 2 2 2
3 3 3 3 3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5 5 5 5 5
6 6 6 6 6 6 6 6 6 6 6 6 6
7 7 7 7 7 7 7 7 7 7 7 7 7
8 8 8 8 8 8 8 8 8 8 8 8 8
9 9 9 9 9 9 9 9 9 9 9 9 9

Data & Labels



0
1
2
3
4
5
6
7
8
9





Exemple applicatif du supervisé

Données :

- Photos de la route
- Décision du conducteur

Au moins 921600 inputs si
photo 1280x720p

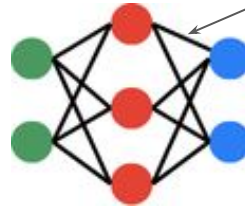




Le réseau de neurones

n couches :

- **1** couche d'entrée
- **$n-2$** couches cachées
- **1** couche de sortie



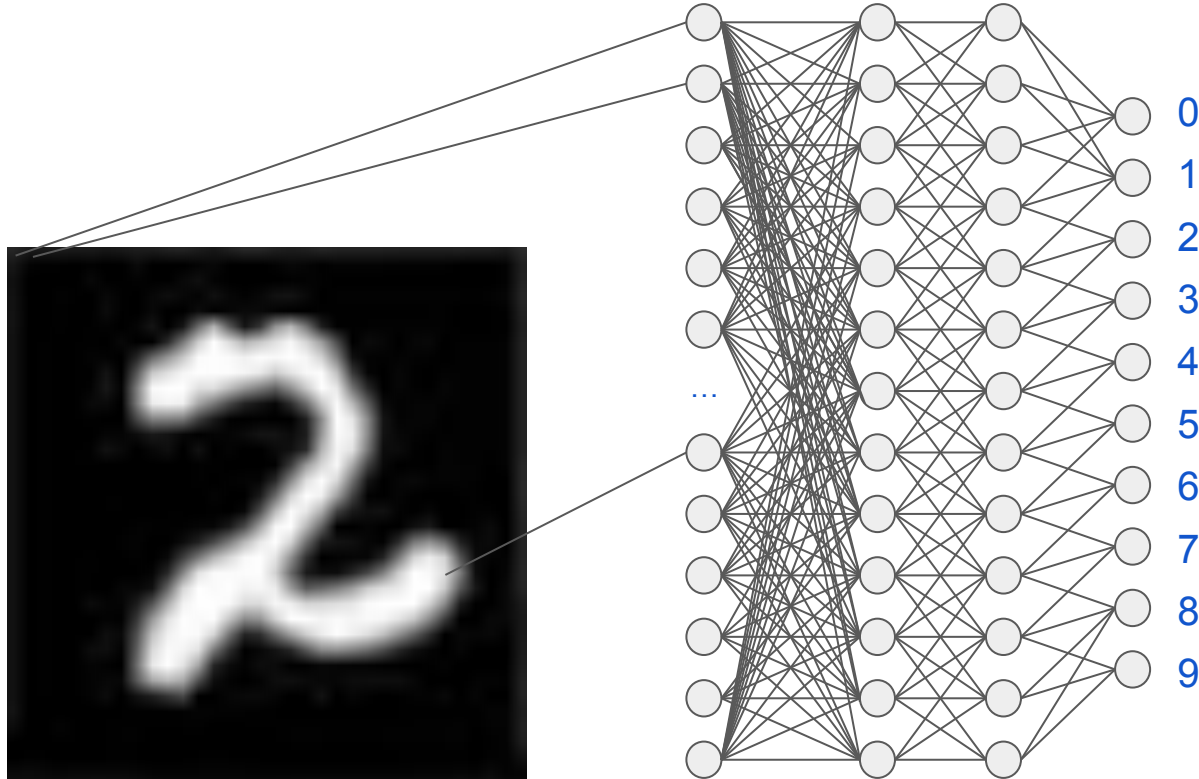
0.38

Chaque couche peut avoir autant de neurones que nécessaire.

Chaque lien entre les neurones représente une valeur décimale utilisée par le neurone à sa droite.



Le réseau de neurones

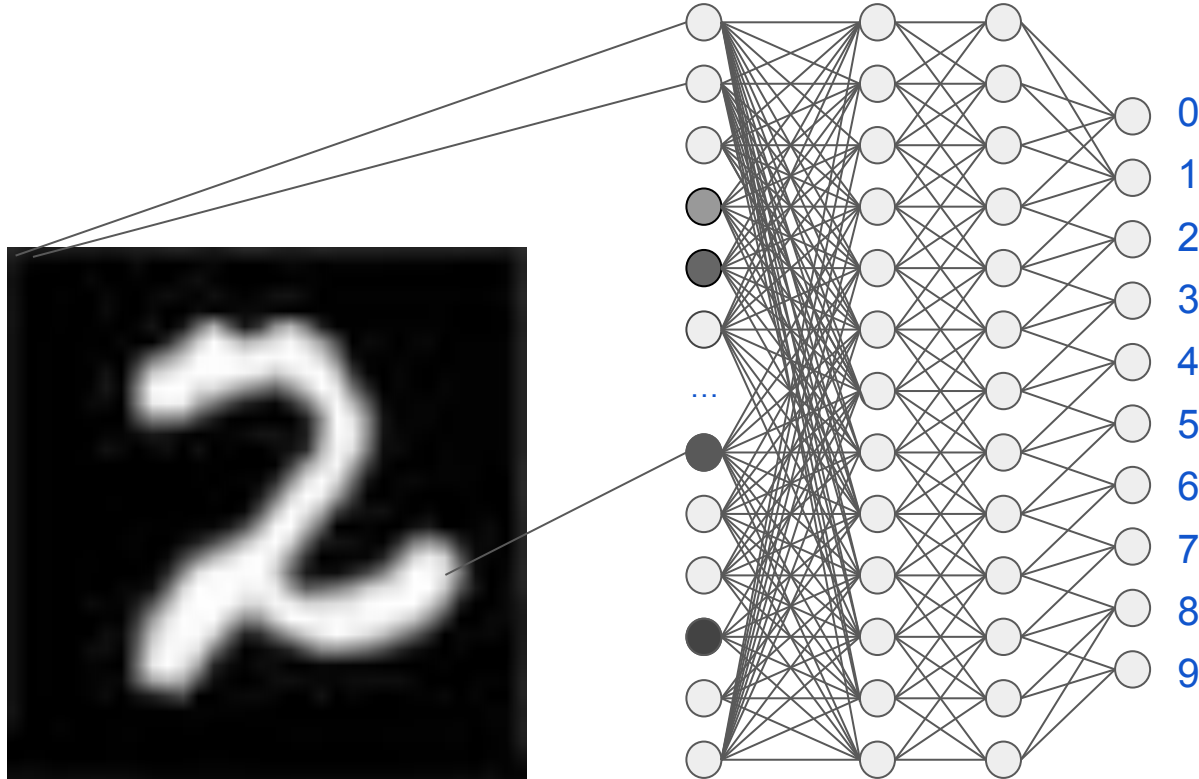


Exemple

- Chaque pixel de l'image possède une valeur de gris.
- Chaque neurone de la couche d'entrée peut prendre cette valeur de gris en entrée.



Le réseau de neurones

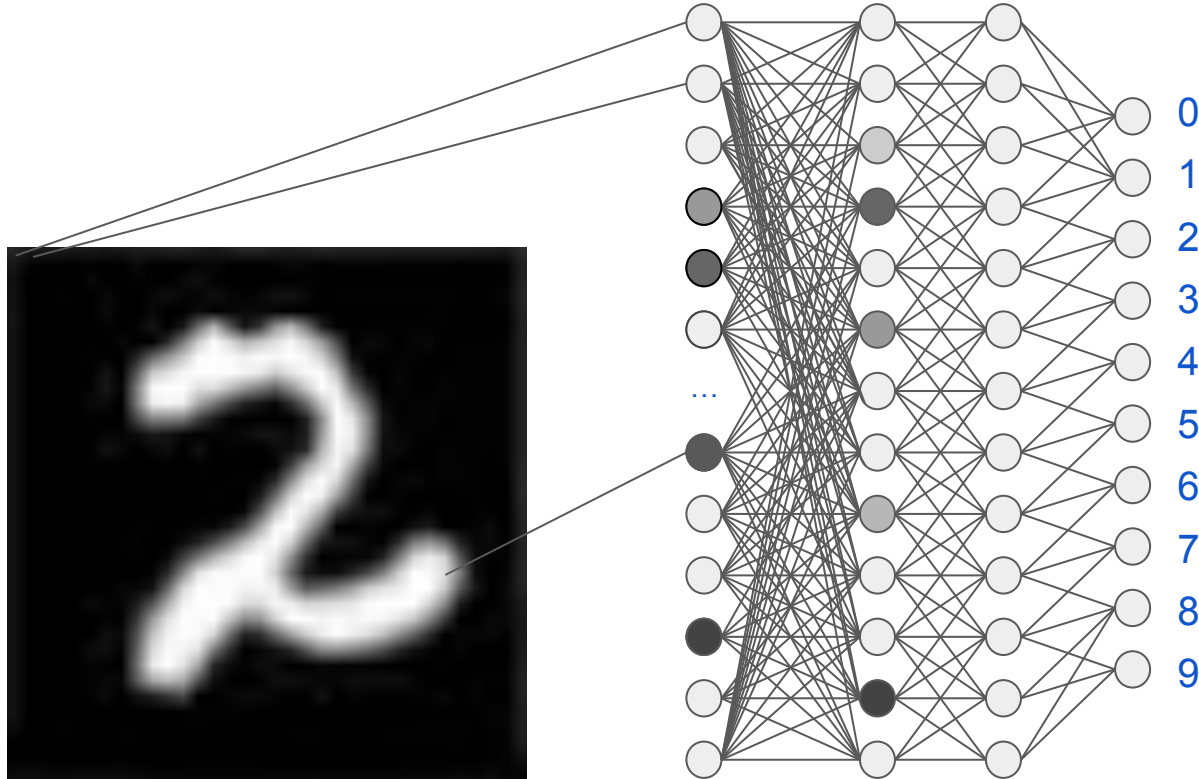


Exemple

- Chaque pixel de l'image possède une valeur de gris.
- Chaque neurone de la couche d'entrée peut prendre cette valeur de gris en entrée.



Le réseau de neurones

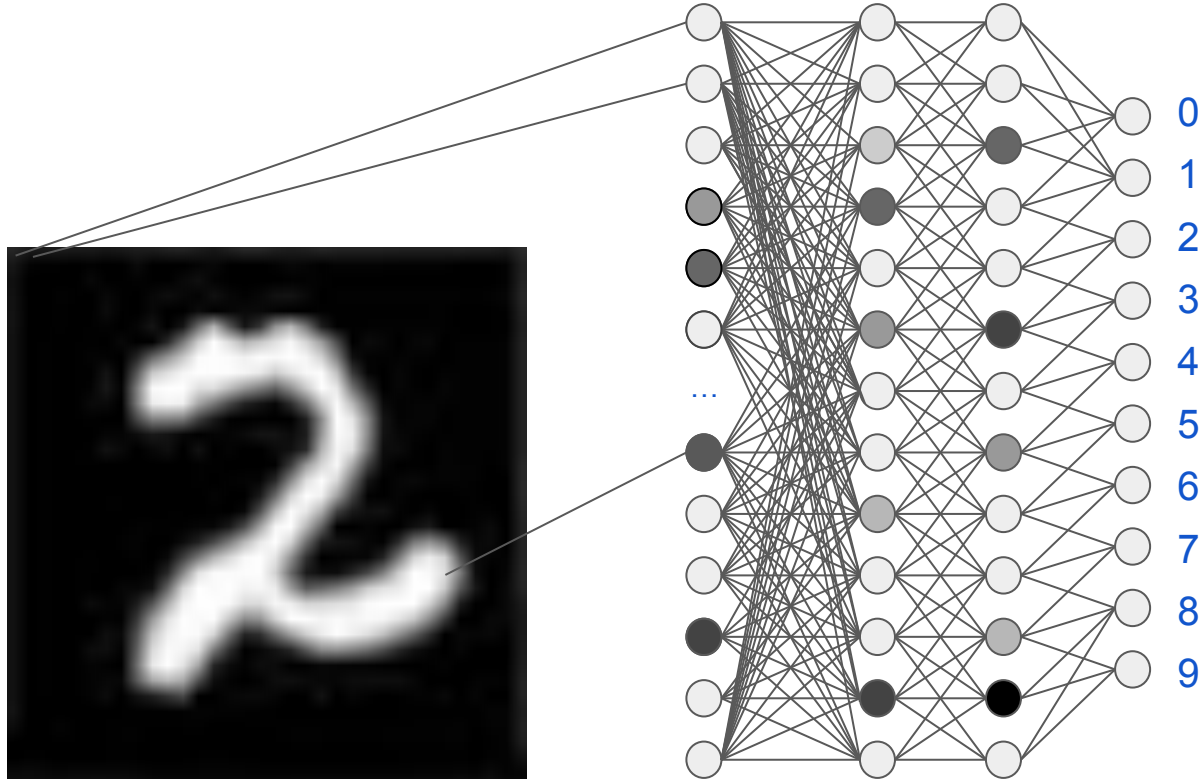


Exemple

- Chaque pixel de l'image possède une valeur de gris.
- Chaque neurone de la couche d'entrée peut prendre cette valeur de gris en entrée.



Le réseau de neurones

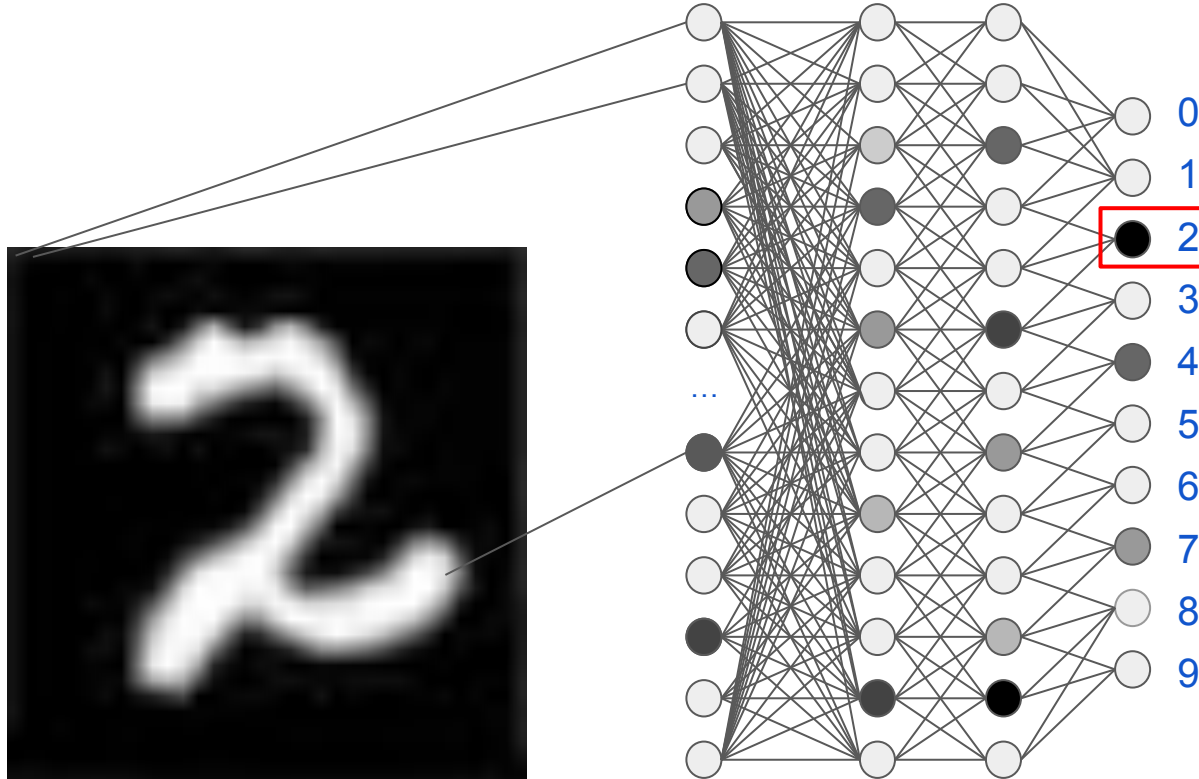


Exemple

- Chaque pixel de l'image possède une valeur de gris.
- Chaque neurone de la couche d'entrée peut prendre cette valeur de gris en entrée.



Le réseau de neurones



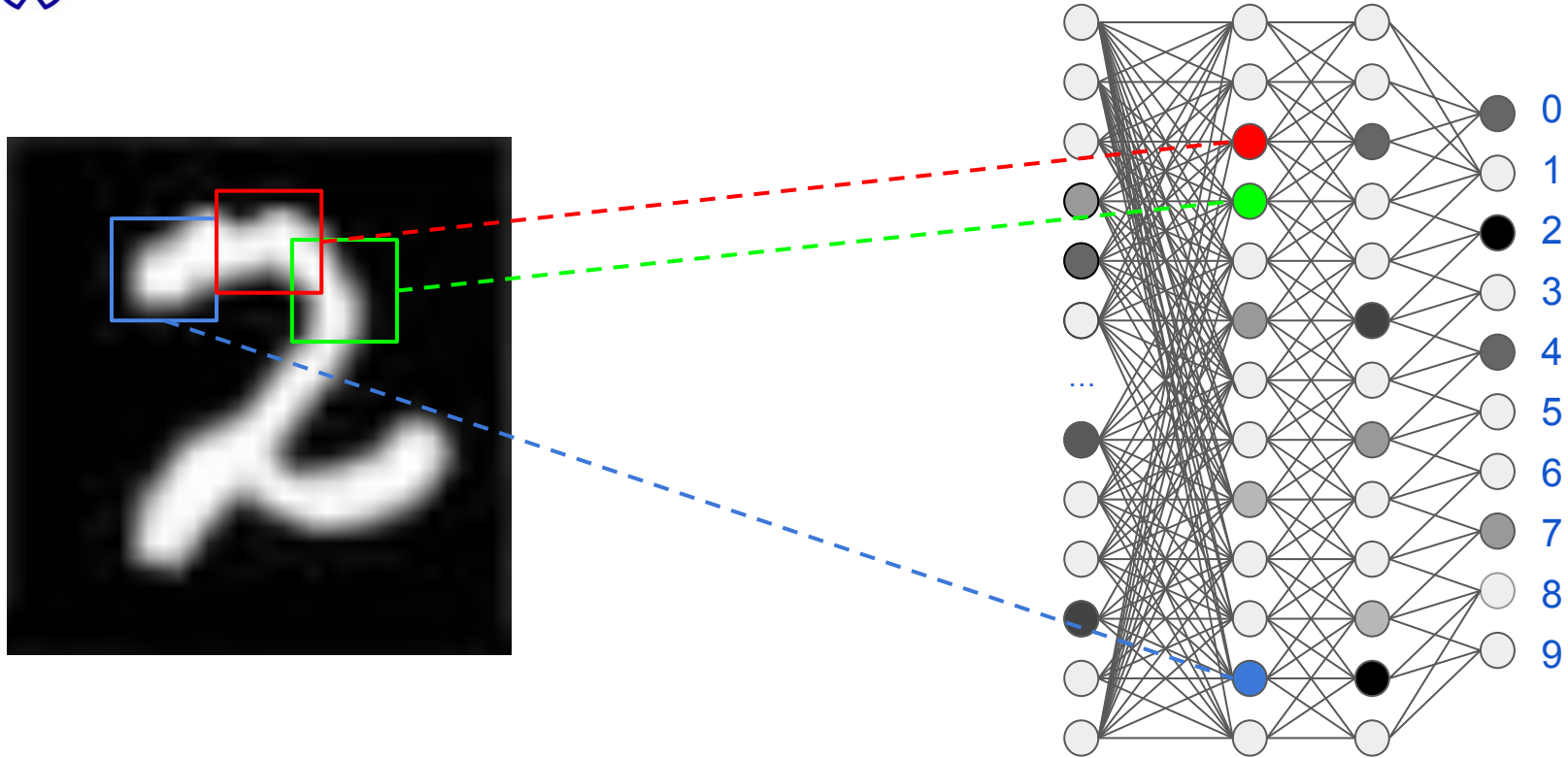
Exemple

- Chaque pixel de l'image possède une valeur de gris.
- Chaque neurone de la couche d'entrée peut prendre cette valeur de gris en entrée.

Chaque neurone représente des parties de l'image à représenter



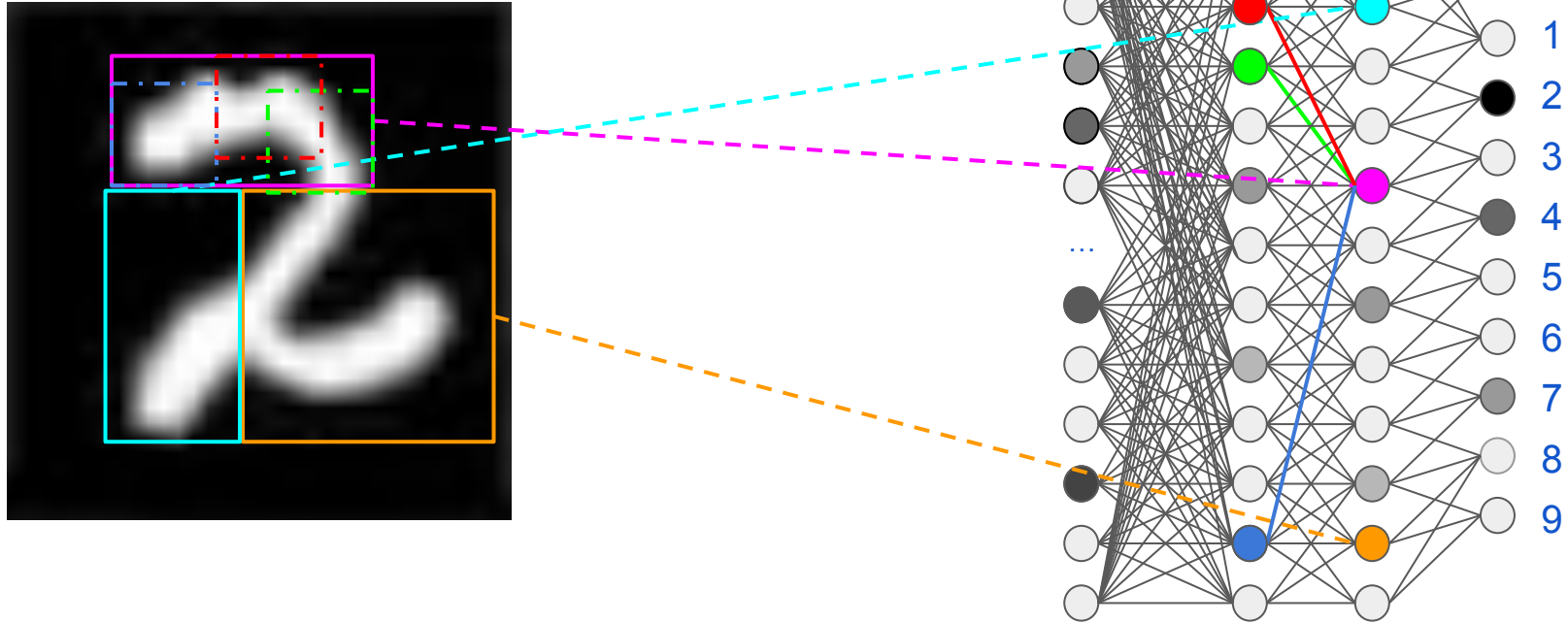
Plus la couche est sur la fin du réseau, plus les représentations sont de haut niveau



Chaque neurone représente des parties de l'image à représenter



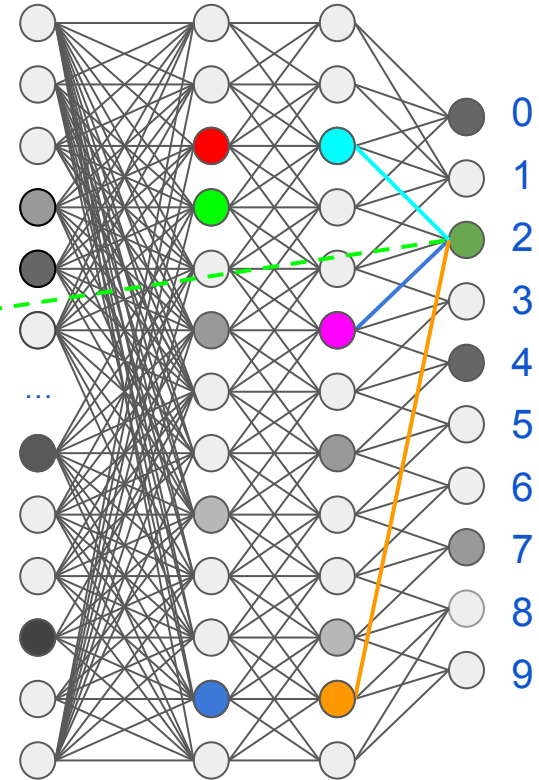
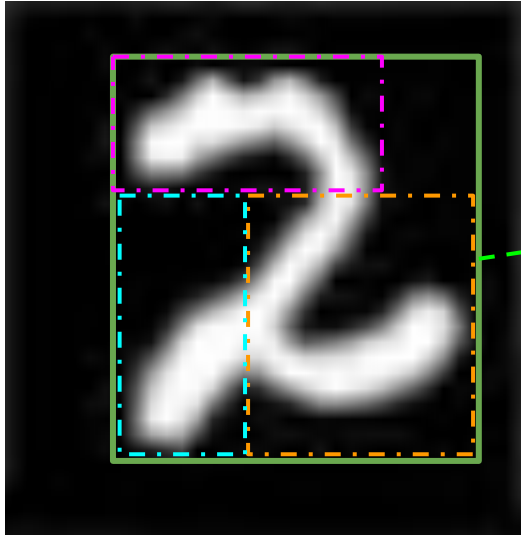
Plus la couche est sur la fin du réseau, plus les représentations sont de haut niveau



Chaque neurone représente des parties de l'image à représenter



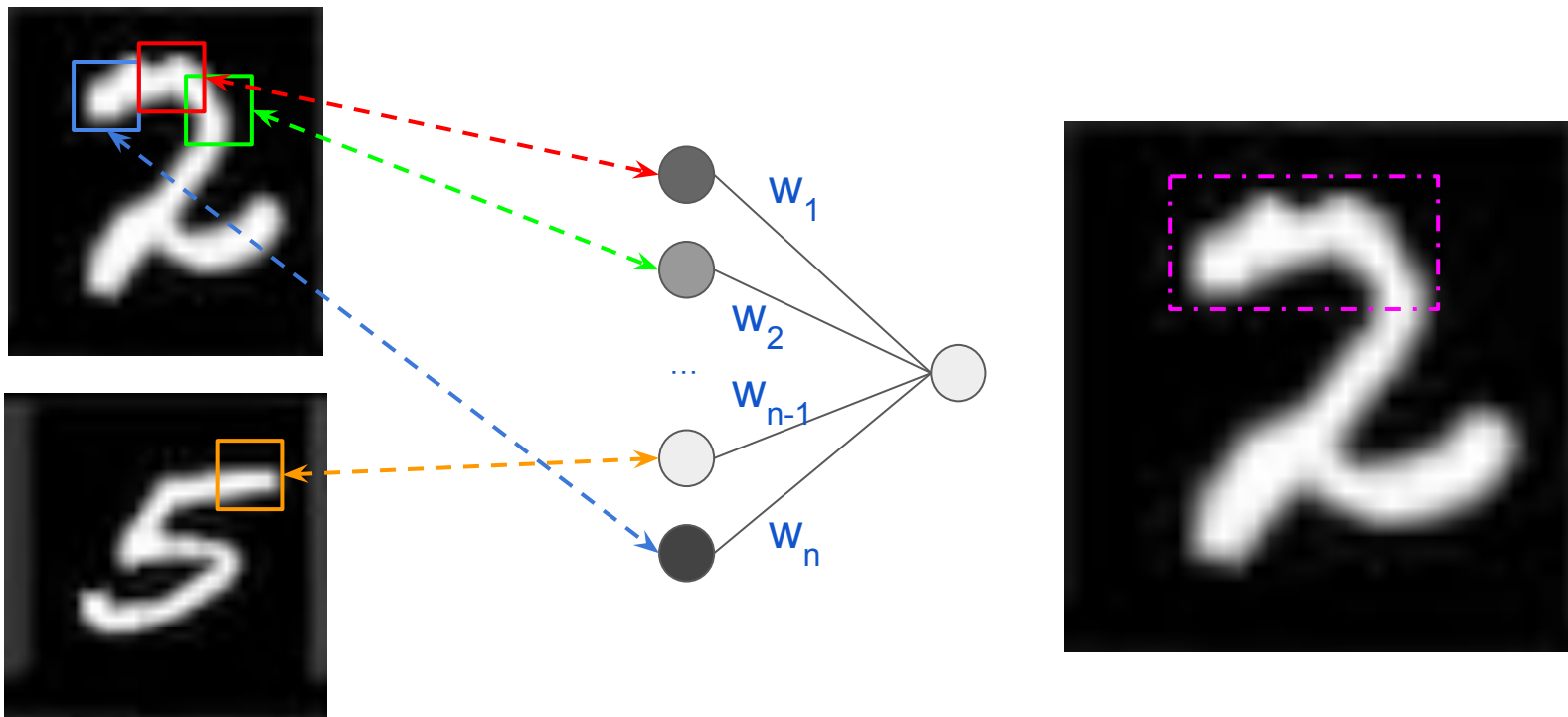
Plus la couche est sur la fin du réseau, plus les représentations sont de haut niveau





Reconnaissance des patterns dans l'image

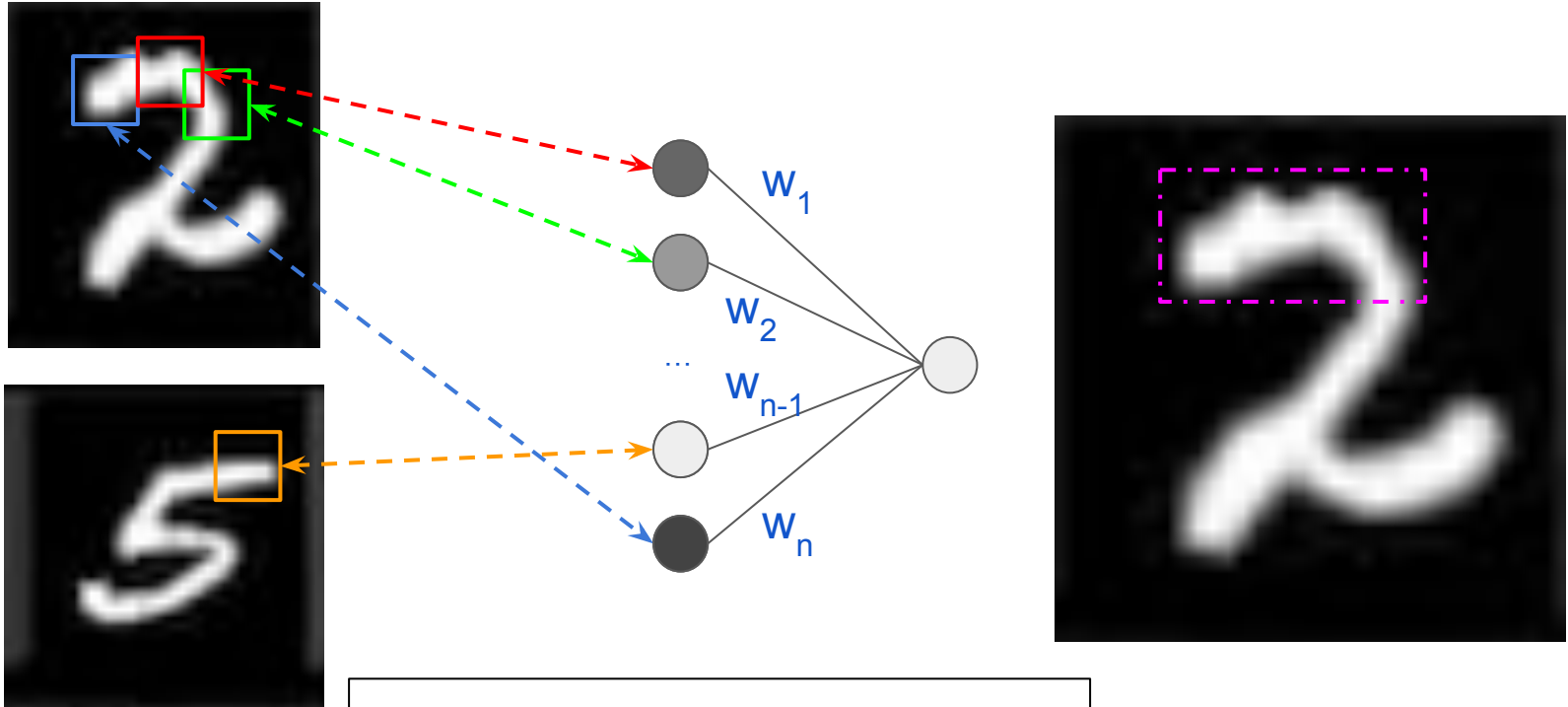
Quels paramètres doivent être utilisés pour que le neurone de droite reconnaisse la zone rose ?





Reconnaissance des patterns dans l'image

Quels paramètres doivent être utilisés pour que le neurone de droite reconnaisse la zone rose ?



$$w_1 a_1 + w_2 a_2 + \dots + w_{n-1} a_{n-1} + w_n a_n$$

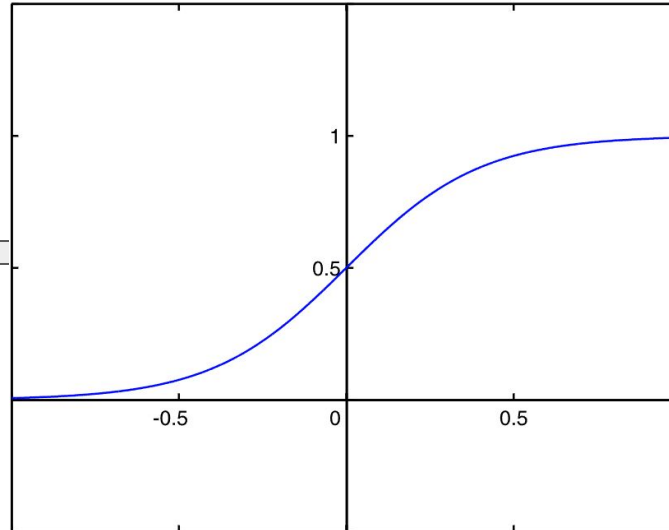


Les fonctions d'activation

Normaliser les activations des neurones

$$\sigma(w_1 a_1 + w_2 a_2 + \dots + w_{n-1} a_{n-1} + w_n a_n)$$

$$\sigma(\sum w_i a_i)$$



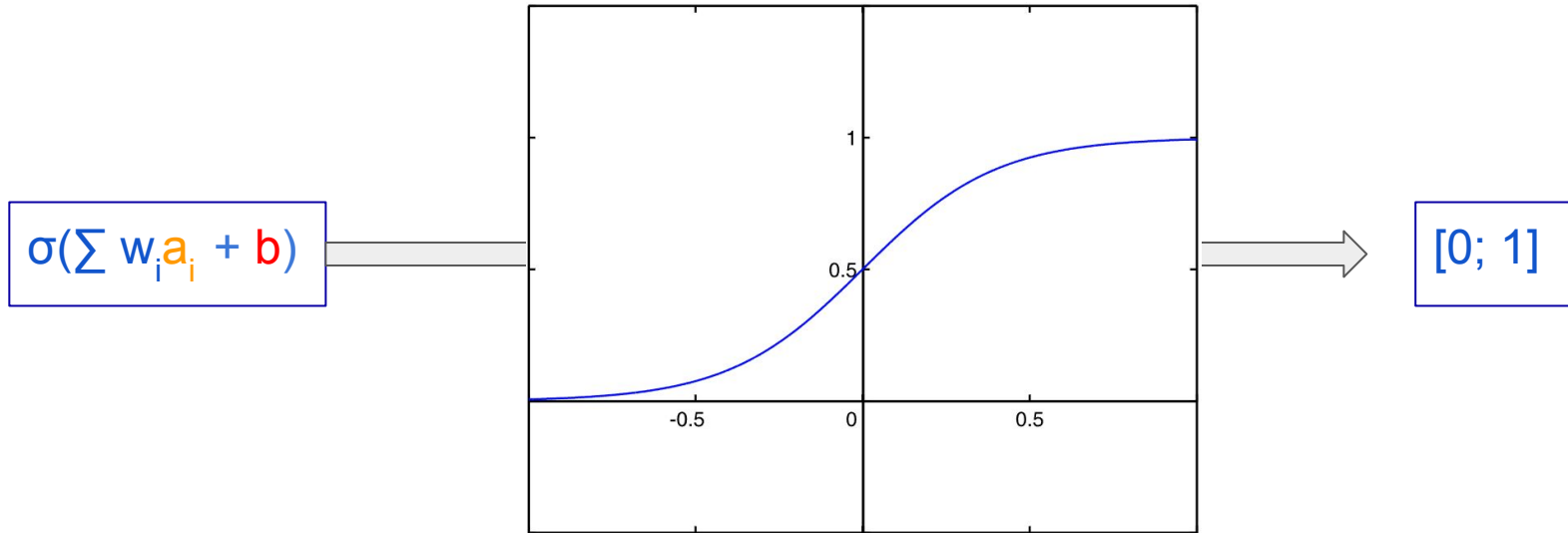
[0; 1]



Les biais

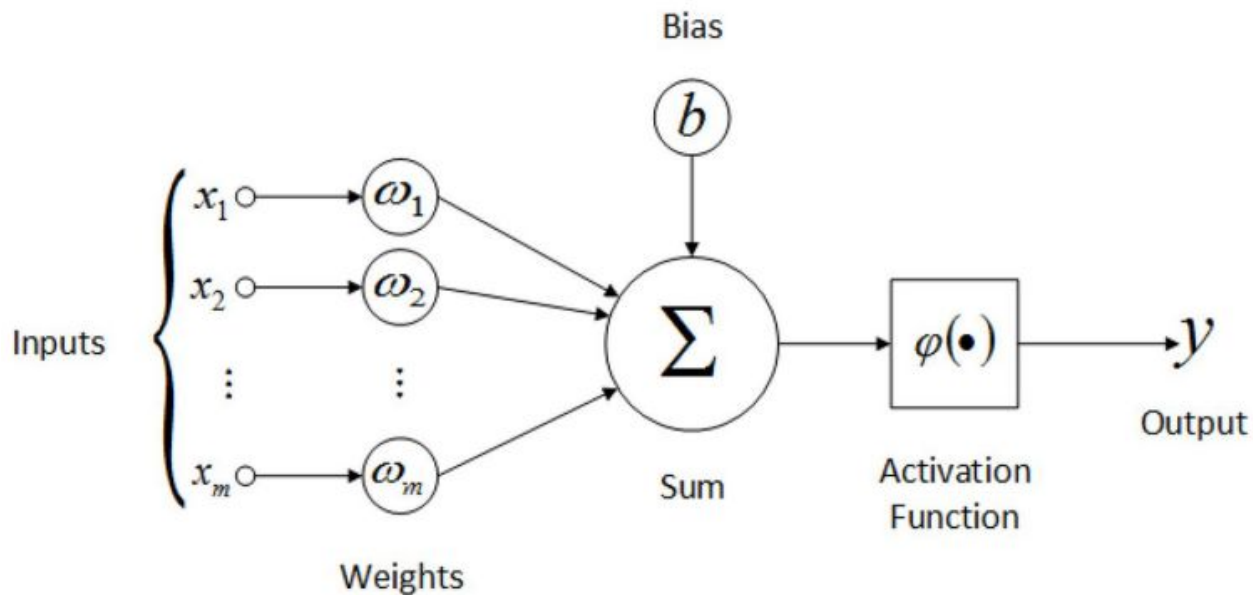
Que faire si l'activation d'un neurone n'apporte rien au résultat ?

$$\sigma(w_1 a_1 + w_2 a_2 + \dots + w_{n-1} a_{n-1} + w_n a_n + b)$$





L'activité d'un neurone en bref





Le réseau de neurones

Couche 1 :

$784 \times 13 \text{ poids} + 13 \text{ biais}$

Couche 2 :

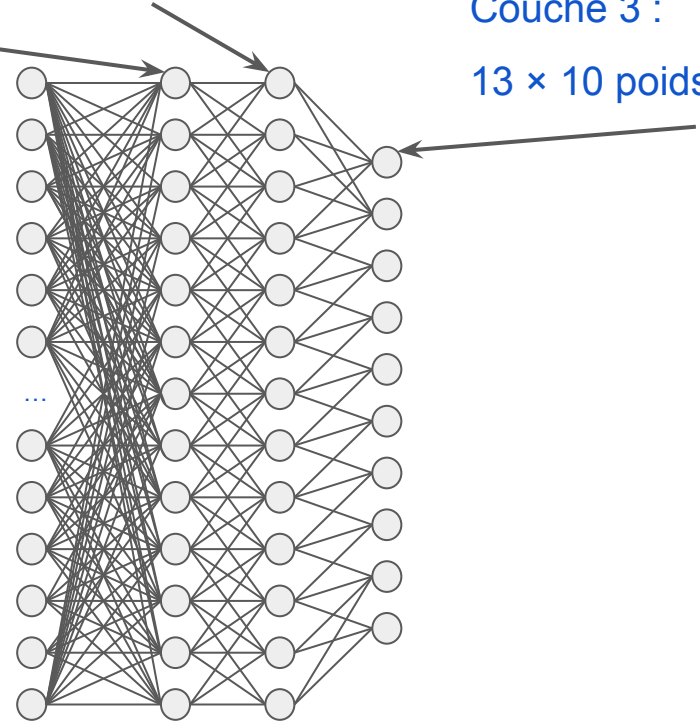
$13 \times 13 \text{ poids} + 13 \text{ biais}$

Couche 3 :

$13 \times 10 \text{ poids} + 10 \text{ biais}$

Chaque neurone possède un ensemble de poids et un biais !

Ce réseau a 10491 poids !





Le réseau de neurones

Couche 1 :

$784 \times 13 \text{ poids} + 13 \text{ biais}$

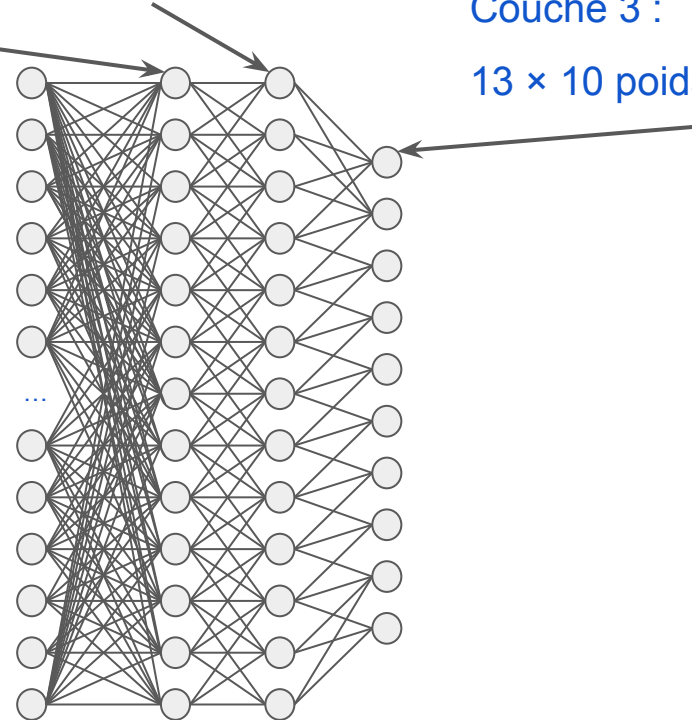
Couche 2 :

$13 \times 13 \text{ poids} + 13 \text{ biais}$

Couche 3 :

$13 \times 10 \text{ poids} + 10 \text{ biais}$

Apprentissage = trouver la valeur des poids et des biais dans le réseau

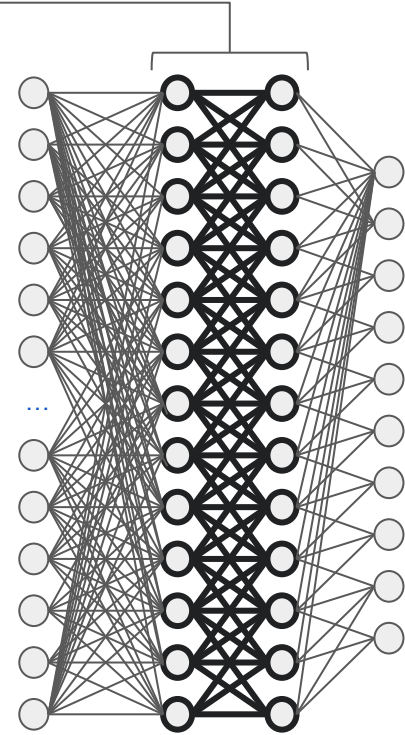




Apprentissage

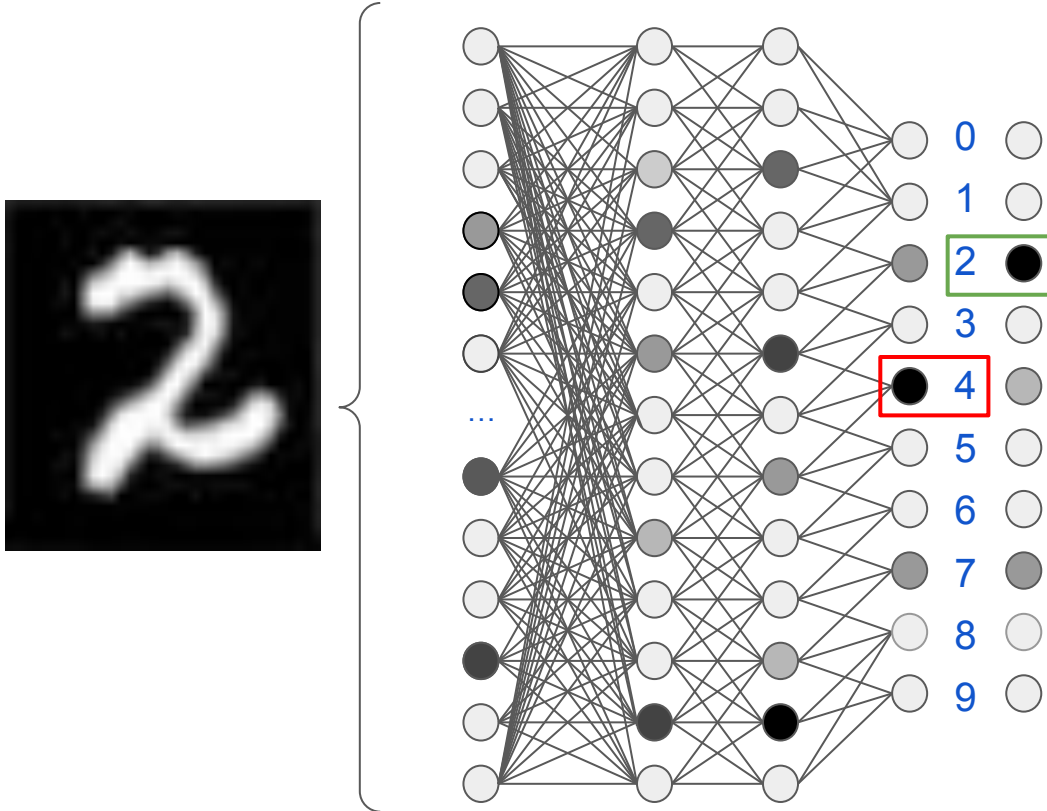
$$\sigma(w_1 a_1 + w_2 a_2 + \dots + w_{n-1} a_{n-1} + w_n a_n + b)$$

$$\sigma \left(\begin{bmatrix} w_{0,0} & w_{0,1} & \dots & w_{0,n} \\ w_{1,0} & w_{1,1} & \dots & w_{1,n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{k,0} & w_{k,1} & \dots & w_{k,n} \end{bmatrix} \begin{bmatrix} a_0^{(0)} \\ a_1^{(0)} \\ \vdots \\ a_n^{(0)} \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_n \end{bmatrix} \right)$$



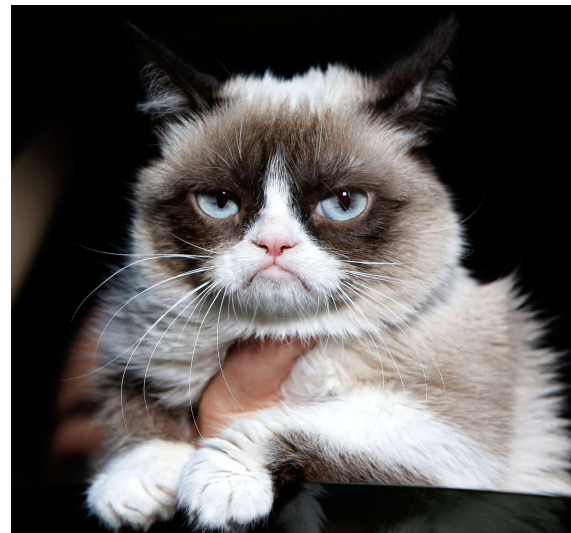
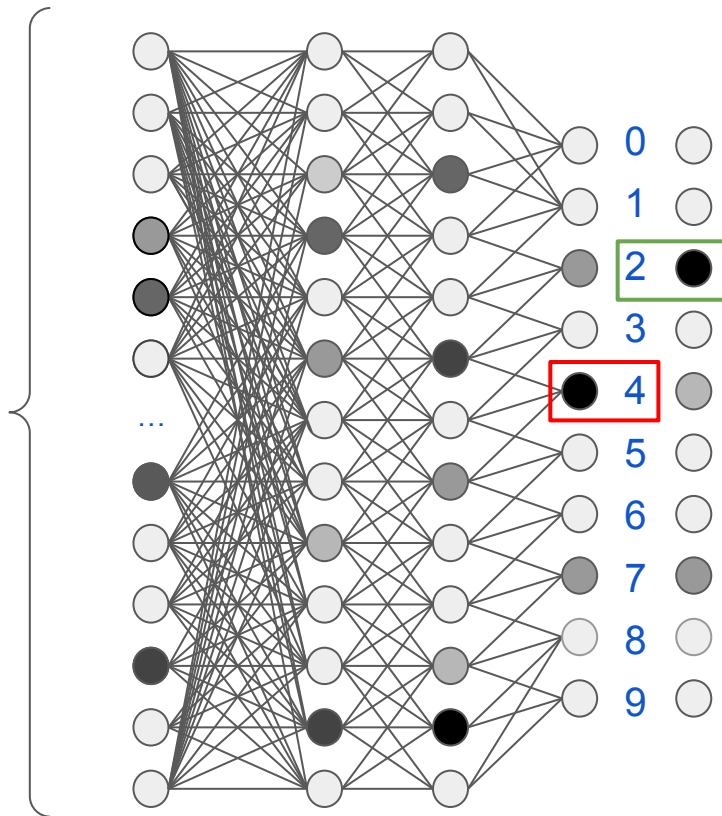


Rétropropagation du gradient





Rétropropagation du gradient





Rétropropagation du gradient

Comment corriger le réseau de neurones en cas d'erreur ?



Activation

- | | | |
|------|----------------------------------|---|
| 0.02 | ☐ | 0 |
| 0.03 | ☐ | 1 |
| 0.21 | ☐ | 2 |
| 0.01 | ☐ | 3 |
| 0.98 | ☒ | 4 |
| 0.05 | ☐ | 5 |
| 0.08 | ☐ | 6 |
| 0.51 | ☐ | 7 |
| 0.01 | ☐ | 8 |
| 0.02 | ☐ | 9 |



Rétropropagation du gradient

Comment corriger le réseau de neurones en cas d'erreur ?



Répéter sur plein de données !

Σ

Attendu Activation

(0.00	-	0.02) ²	☐	0
(0.00	-	0.03) ²	☐	1
(1.00	-	0.21) ²	☒	2
(0.00	-	0.01) ²	☐	3
(0.00	-	0.98) ²	☒	4
(0.00	-	0.05) ²	☐	5
(0.00	-	0.08) ²	☐	6
(0.00	-	0.51) ²	☒	7
(0.00	-	0.01) ²	☐	8
(0.00	-	0.02) ²	☐	9

= 1.85

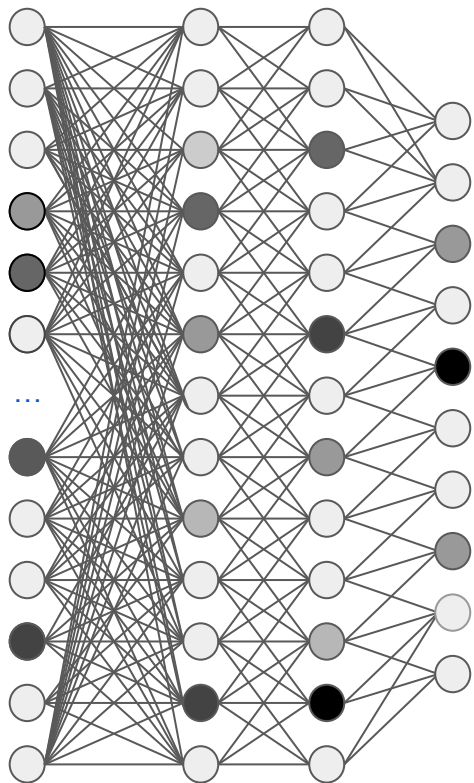
Fonction de coût à minimiser :

$$J = \frac{1}{2} * \sum (y - \hat{y})^2$$



Rétropropagation du gradient

Comment corriger le réseau de neurones en cas d'erreur ?



Attendu *Activation*

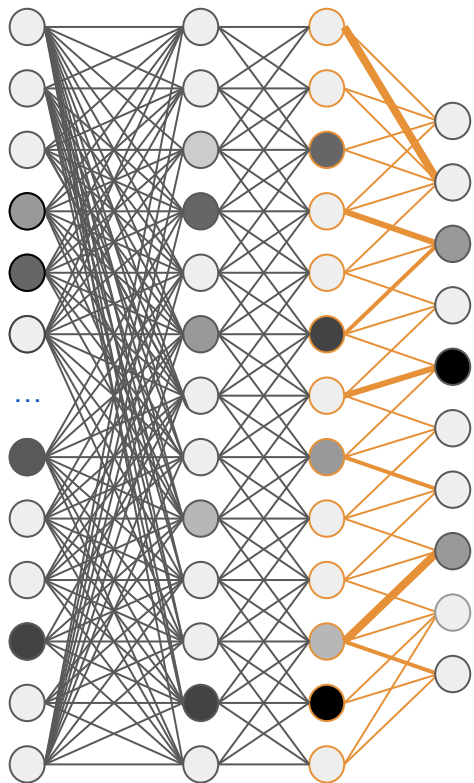
$(0.00$	-	$0.02)^2$		0
$(0.00$	-	$0.03)^2$		1
$(1.00$	-	$0.21)^2$		2
$(0.00$	-	$0.01)^2$		3
$(0.00$	-	$0.98)^2$		4
$(0.00$	-	$0.05)^2$		5
$(0.00$	-	$0.08)^2$		6
$(0.00$	-	$0.51)^2$		7
$(0.00$	-	$0.01)^2$		8
$(0.00$	-	$0.02)^2$		9

= 1.85



Rétropropagation du gradient

Comment corriger le réseau de neurones en cas d'erreur ?



Attendu *Activation*

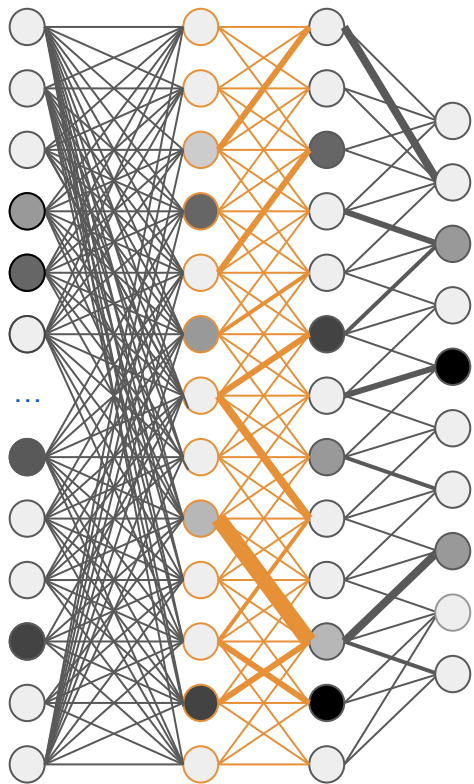
$(0.00$	-	$0.02)^2$		0
$(0.00$	-	$0.03)^2$		1
$(1.00$	-	$0.21)^2$		2
$(0.00$	-	$0.01)^2$		3
$(0.00$	-	$0.98)^2$		4
$(0.00$	-	$0.05)^2$		5
$(0.00$	-	$0.08)^2$		6
$(0.00$	-	$0.51)^2$		7
$(0.00$	-	$0.01)^2$		8
$(0.00$	-	$0.02)^2$		9

= 1.85



Rétropropagation du gradient

Comment corriger le réseau de neurones en cas d'erreur ?



Attendu *Activation*

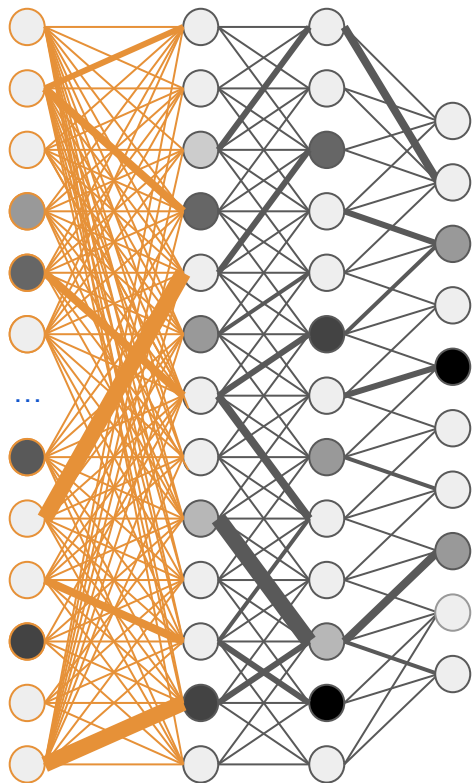
$(0.00$	-	$0.02)^2$		0
$(0.00$	-	$0.03)^2$		1
$(1.00$	-	$0.21)^2$		2
$(0.00$	-	$0.01)^2$		3
$(0.00$	-	$0.98)^2$		4
$(0.00$	-	$0.05)^2$		5
$(0.00$	-	$0.08)^2$		6
$(0.00$	-	$0.51)^2$		7
$(0.00$	-	$0.01)^2$		8
$(0.00$	-	$0.02)^2$		9

= 1.85



Rétropropagation du gradient

Comment corriger le réseau de neurones en cas d'erreur ?



Σ

Attendu *Activation*

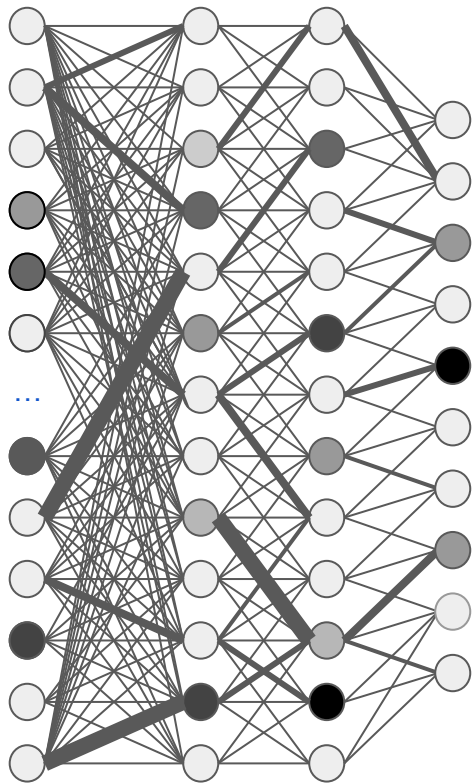
$(0.00$	-	$0.02)^2$		0
$(0.00$	-	$0.03)^2$		1
$(1.00$	-	$0.21)^2$		2
$(0.00$	-	$0.01)^2$		3
$(0.00$	-	$0.98)^2$		4
$(0.00$	-	$0.05)^2$		5
$(0.00$	-	$0.08)^2$		6
$(0.00$	-	$0.51)^2$		7
$(0.00$	-	$0.01)^2$		8
$(0.00$	-	$0.02)^2$		9

= 1.85



Rétropropagation du gradient

Comment corriger le réseau de neurones en cas d'erreur ?



Attendu *Activation*

$(0.00$	-	$0.02)^2$		0
$(0.00$	-	$0.03)^2$		1
$(1.00$	-	$0.21)^2$		2
$(0.00$	-	$0.01)^2$		3
$(0.00$	-	$0.98)^2$		4
$(0.00$	-	$0.05)^2$		5
$(0.00$	-	$0.08)^2$		6
$(0.00$	-	$0.51)^2$		7
$(0.00$	-	$0.01)^2$		8
$(0.00$	-	$0.02)^2$		9

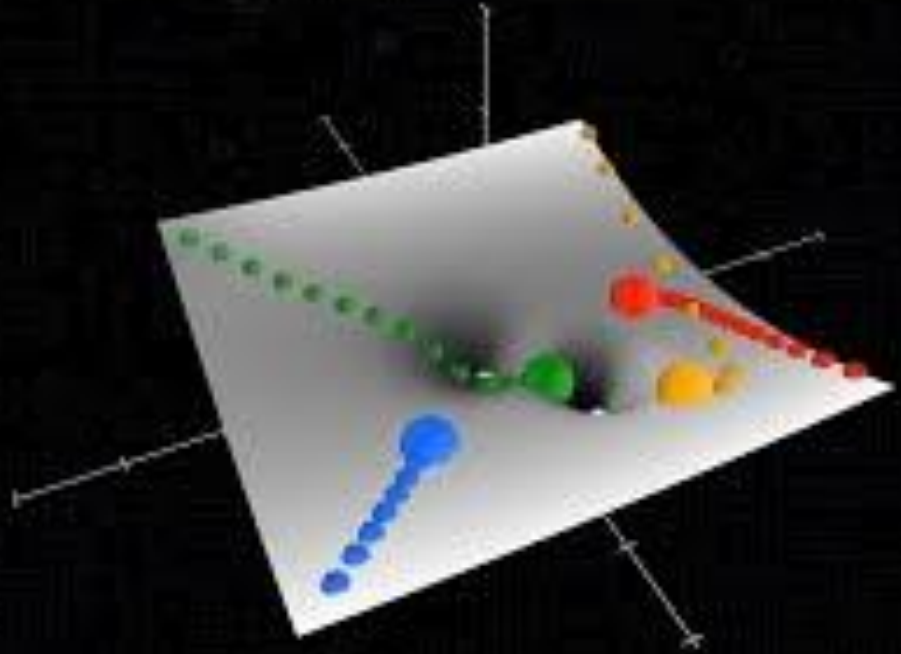
= 1.85



Descente de gradient

Optimizers in Deep Learning

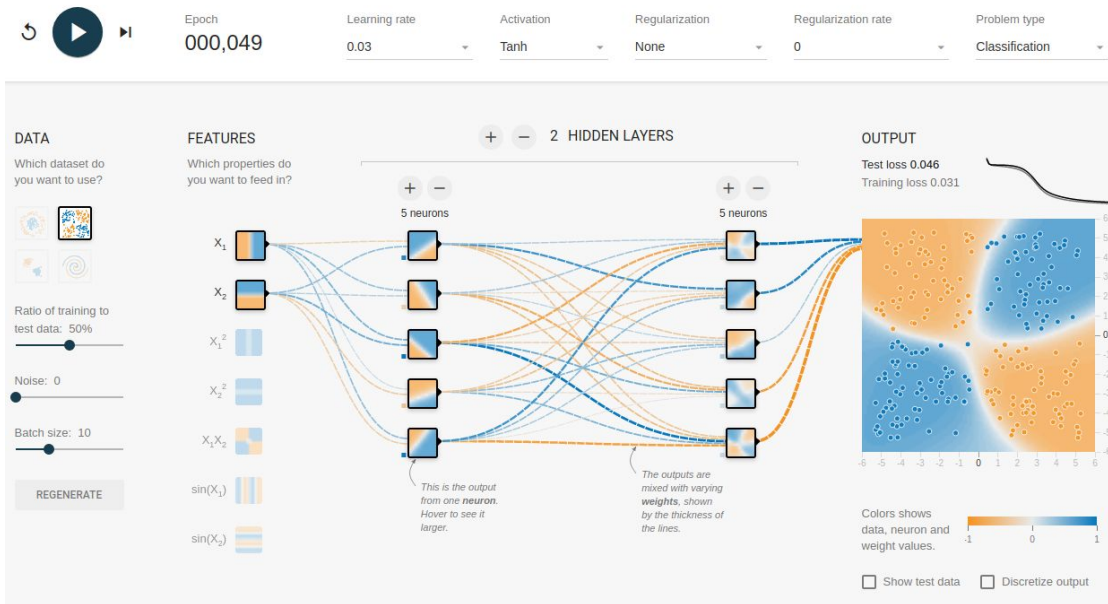
- ⚙️ **SGD**
- ⚙️ **Momentum**
- ⚙️ **RMSProp**
- ⚙️ **Adam**





Tensorflow Playground

Tinker With a **Neural Network** Right Here in Your Browser.
Don't Worry, You Can't Break It. We Promise.



<https://playground.tensorflow.org/>



Tensorflow Playground

Exercices:

- **Tâche 1:** le modèle donné combine nos deux caractéristiques d'entrée en un seul neurone. Ce modèle va-t-il apprendre des non-linéarités ? Exécutez-le pour confirmer votre supposition.
- **Tâche 2 :** essayez d'augmenter le nombre de neurones dans la couche cachée de 1 à 2, et essayez également de passer d'une activation linéaire à une activation non linéaire comme la fonction ReLU. Pouvez-vous créer un modèle capable d'apprendre des non-linéarités ? Peut-il modéliser les données efficacement ?
- **Tâche 4:** poursuivez l'expérimentation en ajoutant ou en supprimant des couches et des neurones cachés par couche. N'hésitez pas non plus à modifier les taux d'apprentissage, la régularisation et d'autres paramètres d'apprentissage. Quel est le *plus petit* nombre de neurones et de couches qui donne une loss de 0,177 ou moins ?



Bilan de l'activité

Activité 1, 2, Tous (1mn, 2mn, 4mn)